



Surprisingly Fast AI-Generated Kernels We Didn't Mean to Publish (Yet)

Authors: Anne Ouyang and Azalia Mirhoseini and Percy Liang

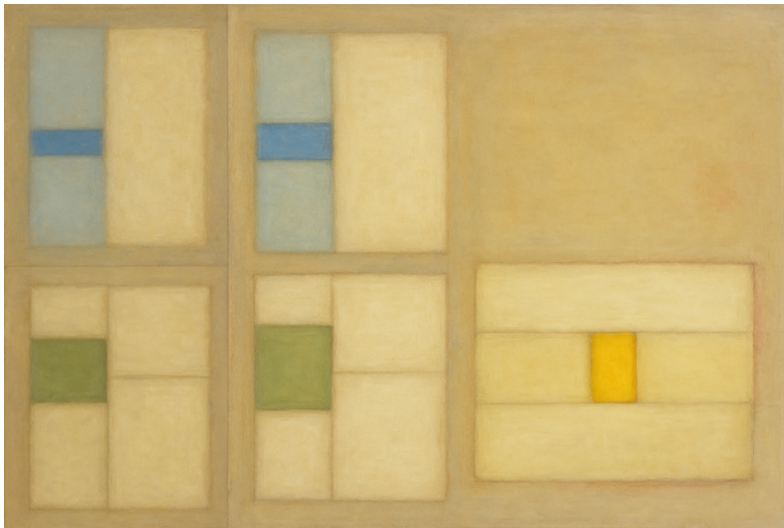
TL;DR

We have some very fast AI-generated kernels in pure CUDA-C without using libraries and DSLs such as CUTLASS and Triton. They are performing close to or in some cases even beating the standard expert-optimized production kernels shipped in PyTorch. Some of our highlighted results:

- **Matmul (FP32): 101.3%** performance of FP32 torch.matmul; problem size: 4096x4096 square matrices
- **Conv2D: 179.9%** performance of FP32 torch.nn.Conv2D; problem size: (100, 3, 224, 224) input tensor, conv(in_channels=3, out_channels=96, kernel_size=11, stride=4, padding=2)
- **Softmax: 111.8%** performance of FP32 torch.softmax; problem size: (4096, 65536) input tensor
- **LayerNorm: 484.4%** performance of FP32 torch.nn.LayerNorm; problem size: (16, 64, 256, 256) input tensor

- **Conv2D + ReLU + MaxPool: 290.1%** performance of FP32 torch reference, 189.0% performance of FP32 torch.compile() reference; problem size: (100, 3, 224, 224) input tensor, conv(in_channels=3, out_channels=96, kernel_size=11, stride=4, padding=2), maxpool(kernel_size=3, stride=2)

(Our results are benchmarked on an Nvidia L40S GPU, and % performance is defined as reference time divided by generated kernel time)



"Untitled" by DALL·E (2025). (Digital pigment on virtual canvas)

From the MMA collection

Intro

We started with the goal of generating synthetic data to train better kernel generation models. Somewhere along the way the unexpected happened: the test-time only synthetic data generation itself started producing *really* good optimized PyTorch kernels beating or performing close to human expert generated features, which were previously considered to be challenging. We decided to write this blog post early and share our findings. The point of this blog post isn't about a novel methodology; in fact, our synthetic data generation design is simple, and what's surprising is that it is already showing promise.

In this post, we're sharing the method, five optimized kernels (4 foundational ML



Center for
Research on
Foundation
Models



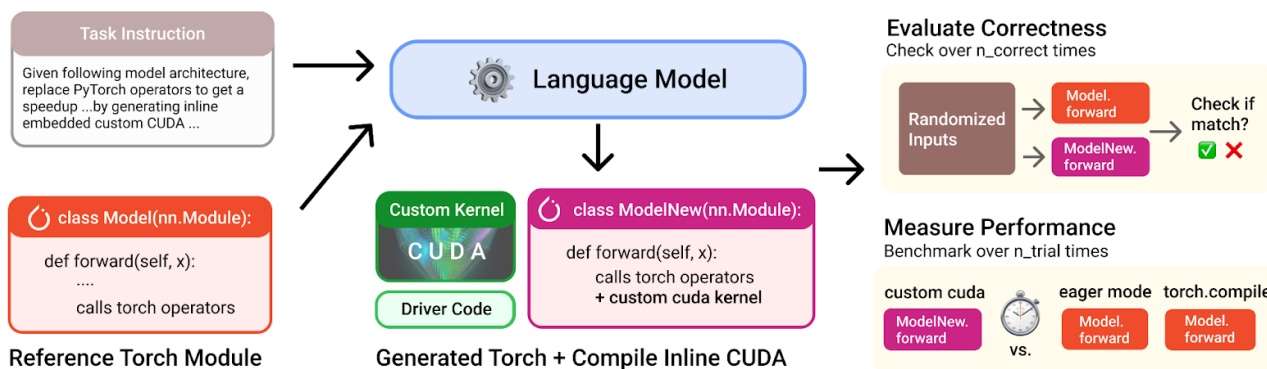
Stanford University
Human-Centered
Artificial Intelligence



operators + 1 fused kernel of an AlexNet block), an example optimization trajectory, and some takeaways and thoughts on what this might mean for performant kernel generation. Consider this a first step in what's next.

Method

We're using the [KernelBench](#) (a benchmark for AI based kernel generation that we released in December 2024) task setup: given torch code, the LLM writes custom kernels to replace the torch operators with the goal of getting a speedup. Consistent with the original KernelBench design, the reference code is in the default FP32, and given a tolerance threshold ($1e-02$), using lower precision solutions is valid. In addition, each problem in KernelBench has specific sizes since there are many size-specific optimizations, so the benchmark tests for the fastest kernel for the specific problem size, not necessarily a generally fast kernel for any arbitrary problem size. We run both the torch reference code and the generated code, and test for correctness by checking the numerical equality of the two outputs over many random inputs.

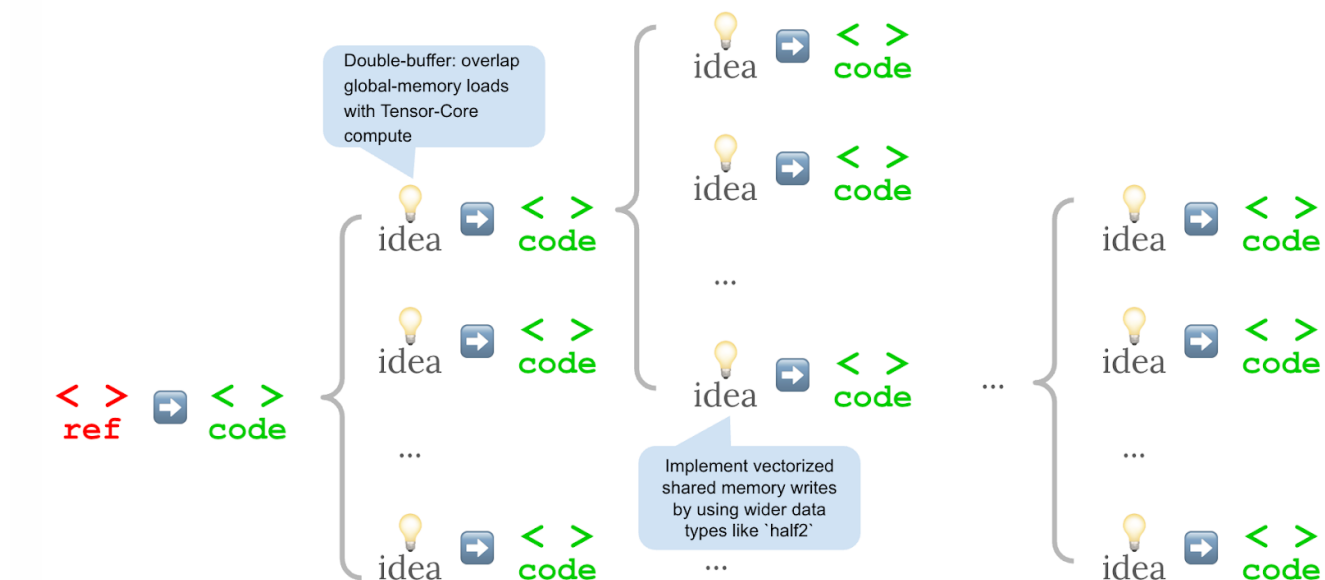


The most common way people scale test-time compute for this problem of optimizing kernels today is through sequential revision, a multi-turn loop where a model incrementally edits a kernel, checks for correctness and performance, then tries again based on the result, either fixing the kernel or try to improve its performance. This loop is intuitive and easy to implement. The model fixes broken kernels, tweaks working ones, and gradually climbs toward something faster.

The main limitation of this approach is the lack of optimization idea diversity. Sequential loops often fall into local minima, revisiting the same classes of transformations or endlessly refining unpromising trajectories. The result is inefficient use of test-time compute and little pressure on the model to generate fundamentally new optimization ideas.

We introduced two key changes to address this:

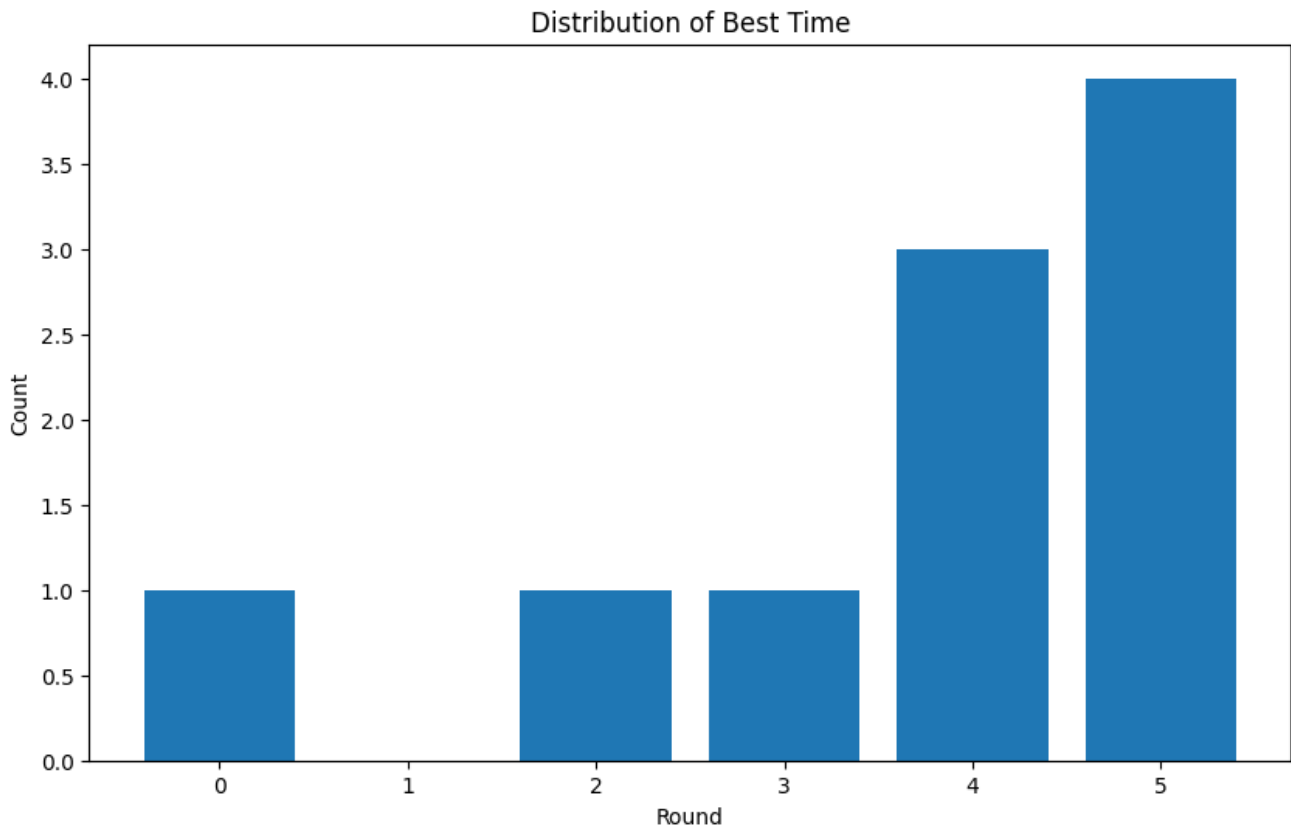
1. Reasoning in natural language about optimization ideas: rather than directly generating new kernels in each step, we generate optimization ideas in natural language conditioned on previously attempted ideas, and realize those ideas into new code variants.
2. Branching at each optimization step: instead of refining a single candidate per step, we fan out such that each idea spawns multiple implementations, and the highest-performing kernels are used to seed the next round (we also keep a bank of good existing kernels for seeding). This unlocks massive parallelism allowing us to explore radically different directions at each turn, rather than getting stuck in a narrow optimization path.



The result is a test-time loop that looks less like “chat with a compiler” in the case of sequential revision, and more like structured exploratory search, guided

by explicit optimization hypotheses and aggressively parallel evaluation.

We ran 10 problems from KernelBench level 1 (and modified the problem sizes to make sure that kernel launch overhead is negligible compared to the overall runtime of the problem). We ran 5 rounds with the OpenAI o3 and Gemini 2.5 Pro models. The plot below shows the distribution of rounds in which the best-performing kernel was first found. Most of the best results emerge in later rounds (out of a total of 5 rounds), with the majority coming in round 4 or 5.



As we scaled up our search, we also found that many high-performing kernels clustered into a few recurring optimization strategies, which also aligns with our experience of writing kernels by hand. The main optimization categories are summarized below:

- **Memory Access Optimization:** improving the efficiency of data movement between different memory hierarchies (global memory, shared memory, registers) and ensuring data is accessed in a way that maximizes bandwidth and minimizes conflicts.

- **Asynchronous Operations & Latency Hiding:** hide the latency of slow operations (like global memory access) by overlapping them with computation or other memory transfers
- **Data Type & Precision Optimization:** using lower-precision data types (like FP16 or BF16) where possible to reduce memory bandwidth requirements, increase cache effectiveness, and potentially leverage specialized hardware units.
- **Compute & Instruction Optimization:** making the arithmetic computations themselves more efficient, reducing instruction count, or leveraging specialized hardware instructions
- **Parallelism & Occupancy Enhancement:** maximize the number of active warps on the Streaming Multiprocessors (SMs) to better hide latencies and improve overall throughput
- **Control Flow & Loop Optimization:** reducing the overhead associated with loops, branches, and indexing calculations

An Example Kernel Optimization Trajectory

Here we show an example optimization trajectory of auto-generated ideas for Conv2D, with torch reference baseline time of **1.41 ms**

Round 0: 7.02 ms, 20.1% of reference

Idea: Given the pytorch code, replace the operation with a CUDA Kernel

Round 1: 7.54 ms, 18.8% of reference

Idea: Exploit the read-only cache by loading invariant tensors with `__ldg`.

Round 2: 3.46 ms, 41.0% of reference

Idea: Convert the convolution to an FP16 Tensor-Core GEMM. *[author comment: this is an algorithmic optimization converting a convolution to an implicit GEMM, which is important for running convolutions efficiently on Tensor Cores]*

Round 3: 3.67 ms, 38.7% of reference

Idea: Double-buffer `cp.async` pipeline that overlaps global-memory loads with

Tensor-Core compute.

Round 4: 3.46 ms, 41.0% of reference

Idea: Given the pytorch code, replace the operation with a CUDA Kernel using implicit matmul. The given GEMM kernel could be helpful.

[author comment: since we know that the optimization involves using GEMM, we seeded the beginning of this round with an existing good GEMM kernel that we generated previously, and this idea is written manually]

Round 5: 1.91 ms, 74.9% of reference

Idea: Precompute and reuse `k\_idx`-decomposed kernel/input indices in shared memory within each K-tile loop to avoid redundant arithmetic.

Round 6: 1.37 ms, 103.6% of reference

Idea: Precomputing and caching N-dimension GEMM indices in shared memory to reduce redundant arithmetic within the input data loading loop.

Round 7: 1.38 ms, 102.9% of reference

Idea: Parallelize CUDA kernel output by using dedicated per-warp shared memory buffers to eliminate warp-wise serialization during writes to global memory.

Round 8: 1.37 ms, 103.6% of reference

Idea: Precompute and cache base input coordinates in shared memory to reduce redundant arithmetic during input data loading.

Round 9: 1.36 ms, 105.1% of reference

Idea: Software-pipeline B-fragment loading to overlap the next B-tile's shared memory reads with the current B-tile's WMMA computations.

Round 10: 1.07 ms, 133.6% of reference

Idea: Reuse precomputed N-dimension GEMM decomposition from shared memory for output address calculation, avoiding redundant and costly division/modulo operations.

Round 11: 1.21 ms, 117.4% of reference

Idea: Remove ``hi/lo`` decomposition in ``half`` WMMA operations, relying on standard FP16 accumulation to improve performance if the resulting accuracy is acceptable.

Round 12: 1.01 ms, 141.2% of reference

Idea: Overlap K-loop global memory loads of ``Asub`` (weights) and ``Bsub`` (inputs) with MMA computation using double buffering, enabled by calculating K-dimension indices on-the-fly within the load stage of the pipeline.

Round 13: 0.795 ms, 179.9% of reference

Idea: Implement vectorized shared memory writes for loading ``Asub_pipe`` and ``Bsub_pipe`` by using wider data types like ``half2``

Final Code Sample

The final code sample for the Conv2D kernel is included in the appendix. It uses advanced CUDA techniques that we find challenging to write ourselves! We also have more example kernels in this [Github repo](#)

Takeaways

Our method echoes a growing theme in AI research: combining strong reasoning with parallel exploration of multiple hypotheses leads to improvements. As some recent work ([AlphaEvolve](#), [Gemini 2.5 Pro Deep Think](#)) highlight, you might not always need massive retraining — sometimes, clever search and branching strategies can unlock scientific innovation and tackle complex problems, and there might be more gains through extensive searching with verifiers.

However, this doesn't mean we shouldn't do further training. On the contrary, our approach also helps generate better synthetic data to improve future model training (this requires more problem instances). So, it's both a powerful test-time scaling method and a step toward smarter, more data-efficient model development.

Finally, what we've demonstrated here is just an early sign of life. The

optimization quality looks promising (it's using many advanced strategies), but there's plenty of room to improve, such as the generation of better optimization ideas, high quality resulting code, as well as applying this to increasingly complicated kernels. Two concrete examples that we are still actively working on improving are:

- FP16 Matmul: 52% performance of torch.matmul
- FP16 Flash Attention: 9% performance of torch.nn.functional.scaled_dot_product_attention

FP32 is less common in modern ML workloads and often less optimized on recent hardware compared to FP16 or BF16, which may partly explain why it's easier to achieve performance gains over PyTorch with FP32 kernels.

Despite the current limitations, we're optimistic. At the time of KernelBench, we couldn't even generate functional versions of these two kernels above, and through searching we've been steadily increasing the performance of flash attention from <1%, and note that we are working with a quite limited search budget here (around 3 million input tokens + 4 million output tokens in total). The progress since then gives us confidence in the potential for continual improvement, and we are excited to keep pushing the frontier of AI to create increasingly better kernels towards the eventual goal of self-improving AI systems.

Thanks

Christopher Rinard, Saman Amarasinghe, and Allen Nie for the helpful discussions; Standard Kernel Co. and Prime Intellect for supporting this work.

Appendix: Fast Conv2D Kernel

```
import torch
import torch.nn as nn
```

```

import torch.nn.functional as F
from torch.utils.cpp_extension import load_inline

conv2d_implicit_gemm_cuda_source = r"""
#include <torch/extension.h>
#include <ATen/cuda/CUDAContext.h> // For at::cuda::getCurrentCUDAStream
#include <mma.h>
#include <cuda_fp16.h>

using namespace nvcuda;

// WMMA tile dimensions
#define WMMA_M 16
#define WMMA_N 16
#define WMMA_K 16

// Skew padding for shared memory to avoid bank conflicts
#define SKEW_HALF 8 // 8 half elements (16 bytes)

// CUDA built-in warpSize is 32 for supported architectures (sm_70+)
// This constant is used for host-side configuration (e.g. blockDim)
#define CUDA_WARP_SIZE_CONST 32

// Threadblock configuration
#define WARPS_PER_BLOCK 8
// THREADS_PER_BLOCK must be evaluatable by host compiler for blockDim
#define THREADS_PER_BLOCK (WARPS_PER_BLOCK * CUDA_WARP_SIZE_CONST)

// Macro-tile dimensions computed by a threadblock
// BLOCK_M_TILES_WMMA * WMMA_M = output channels processed by a block
// BLOCK_N_TILES_WMMA * WMMA_N = output spatial elements processed by
#define BLOCK_M_TILES_WMMA 8
#define BLOCK_N_TILES_WMMA 8

#define TILE_M_PER_BLOCK (BLOCK_M_TILES_WMMA * WMMA_M) // e.g., 8 * 16 = 128
#define TILE_N_PER_BLOCK (BLOCK_N_TILES_WMMA * WMMA_N) // e.g., 8 * 16 = 128

// Vector size for shared memory writes (half2)

```

```

#define VECTOR_SIZE_H2 2

// Struct to hold precomputed N-dimension GEMM indices
struct NDecomposed {
    int ow_eff;
    int oh_eff;
    int n_batch_idx;
    bool isValidPixel; // True if this pixel_idx is within N_gemm bo
    int h_in_base;
    int w_in_base;
};

__global__ void conv2d_implicit_gemm_wmma_kernel(
    const float* __restrict__ input_ptr,    // Input: (N, Cin, Hin,
    const float* __restrict__ weight_ptr,   // Weights: (Cout, Cin,
    const float* __restrict__ bias_ptr,     // Bias: (Cout) or nullp
    float* __restrict__ output_ptr,        // Output: (N, Cout, Ho
    const int N_batch, const int C_in, const int H_in, const int W_i
    const int C_out, const int K_h, const int K_w,
    const int stride_h, const int stride_w,
    const int pad_h, const int pad_w,
    const int H_out, const int W_out,
    const int M_gemm, // C_out
    const int N_gemm, // N_batch * H_out * W_out
    const int K_gemm  // C_in * K_h * K_w
) {
    // Thread identification
    const int warp_id = threadIdx.x / warpSize;           // 0 .. WARP
    const int lane_id = threadIdx.x % warpSize;           // 0 .. 31 (

    // Top-left corner of the macro-tile this block is responsible f
    const int block_row_gemm_start = TILE_M_PER_BLOCK * blockIdx.y;
    const int block_col_gemm_start = TILE_N_PER_BLOCK * blockIdx.x;

    // Shared memory for tiles of A (weights) and B (input/im2col) -
    __shared__ half Asub_pipe[2][TILE_M_PER_BLOCK][WMMA_K + SKEW_HALF
    __shared__ half Bsub_pipe[2][TILE_N_PER_BLOCK][WMMA_K + SKEW_HALF

```

```

// Shared memory for precomputed N-indices
__shared__ NDecomposed n_params_sh[TILE_N_PER_BLOCK];

// Shared memory for output stage (per-warp buffers)
__shared__ float C_shmem_output_buffers[WARPS_PER_BLOCK][WMMA_M][WMMA_N];

// Accumulator fragments per warp.
wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float>
#pragma unroll
for (int i = 0; i < BLOCK_N_TILES_WMMA; ++i) {
    wmma::fill_fragment(acc_frag[i], 0.0f);
}

// Populate n_params_sh once at the beginning of the kernel
if (threadIdx.x < TILE_N_PER_BLOCK) {
    int r_b_tile_idx = threadIdx.x;
    int current_pixel_idx = block_col_gemm_start + r_b_tile_idx;

    if (current_pixel_idx < N_gemm) {
        n_params_sh[r_b_tile_idx].ow_eff = current_pixel_idx % W_out;
        int temp_div_wout = current_pixel_idx / W_out;
        n_params_sh[r_b_tile_idx].oh_eff = temp_div_wout % H_out;
        n_params_sh[r_b_tile_idx].n_batch_idx = temp_div_wout / H_out;
        n_params_sh[r_b_tile_idx].isValidPixel = true;

        n_params_sh[r_b_tile_idx].h_in_base = n_params_sh[r_b_tile_idx].h_in_base;
        n_params_sh[r_b_tile_idx].w_in_base = n_params_sh[r_b_tile_idx].w_in_base;
    } else {
        n_params_sh[r_b_tile_idx].isValidPixel = false;
        n_params_sh[r_b_tile_idx].ow_eff = 0;
        n_params_sh[r_b_tile_idx].oh_eff = 0;
        n_params_sh[r_b_tile_idx].n_batch_idx = 0;
        n_params_sh[r_b_tile_idx].h_in_base = 0;
        n_params_sh[r_b_tile_idx].w_in_base = 0;
    }
}
__syncthreads();

```

```

// Constants for vectorized shared memory loading
// Number of half2 elements along K-dim for a shared memory tile
const int NUM_H2_ELEMENTS_IN_K_DIM = WMMA_K / VECTOR_SIZE_H2;
// Number of thread groups, where each group has NUM_H2_ELEMENTS
// Each group is responsible for loading the K-dimension for one
// iterating over M-rows or N-rows with this step size.
const int NUM_ROW_PROCESSING_GROUPS = THREADS_PER_BLOCK / NUM_H2_

// --- K-Loop Pipelining ---
int num_k_tiles = (K_gemm + WMMA_K - 1) / WMMA_K;

// --- Prologue: Load first k-tile (k_tile_iter = 0) into pipe_i
if (num_k_tiles > 0) {
    int k_tile_start_prologue = 0;
    int current_pipe_idx_prologue = 0;

    // Load Asub_pipe[0] for k_tile_iter = 0
    {
        // This thread is responsible for the 'h2_idx_in_k_dim'
        // in the K-dimension of the shared memory tile.
        int h2_idx_in_k_dim_A = threadIdx.x % NUM_H2_ELEMENTS_IN_K_DIM;
        // Starting 'half' index in shared memory for this half
        int shmem_k_start_for_h2_A = h2_idx_in_k_dim_A * VECTOR_SIZE_H2;

        // Global k-indices for the two half elements.
        int k_global_A_0 = k_tile_start_prologue + shmem_k_start_for_h2_A;
        int k_global_A_1 = k_tile_start_prologue + shmem_k_start_for_h2_A + WMMA_K;

        // Decompose k_global_A_0
        int kw_eff_reg_A_0 = 0, kh_eff_reg_A_0 = 0, ic_eff_reg_A_0 = 0;
        bool is_valid_k_A_0 = (k_global_A_0 < K_gemm);
        if (is_valid_k_A_0) {
            kw_eff_reg_A_0 = k_global_A_0 % K_w;
            int temp_div_kw_A_0 = k_global_A_0 / K_w;
            kh_eff_reg_A_0 = temp_div_kw_A_0 % K_h;
            ic_eff_reg_A_0 = temp_div_kw_A_0 / K_h;
        }
    }
}

```

```

// Decompose k_global_A_1
int kw_eff_reg_A_1 = 0, kh_eff_reg_A_1 = 0, ic_eff_reg_A_1 = 0;
bool is_valid_k_A_1 = (k_global_A_1 < K_gemm);
if (is_valid_k_A_1) {
    kw_eff_reg_A_1 = k_global_A_1 % K_w;
    int temp_div_kw_A_1 = k_global_A_1 / K_w;
    kh_eff_reg_A_1 = temp_div_kw_A_1 % K_h;
    ic_eff_reg_A_1 = temp_div_kw_A_1 / K_h;
}

// This thread belongs to 'm_row_group_id_A'-th group of M-rows
// This group iterates over M-rows of the Asub_pipe tile
int m_row_group_id_A = threadIdx.x / NUM_H2_ELEMENTS_IN_TILE;
for (int r_a_tile_base = m_row_group_id_A; r_a_tile_base < M; r_a_tile_base += M) {
    int oc_idx = block_row_gemm_start + r_a_tile_base;
    float weight_val_0 = 0.0f;
    if (oc_idx < C_out && is_valid_k_A_0) {
        weight_val_0 = weight_ptr[oc_idx * C_in * K_h + ic_eff_reg_A_0 * K_h + kh_eff_reg_A_0 * K_w + kw_eff_reg_A_0];
    }
    float weight_val_1 = 0.0f;
    if (oc_idx < C_out && is_valid_k_A_1) {
        weight_val_1 = weight_ptr[oc_idx * C_in * K_h + ic_eff_reg_A_1 * K_h + kh_eff_reg_A_1 * K_w + kw_eff_reg_A_1];
    }
    half2* smem_ptr_h2_A = reinterpret_cast<half2*>(&Asub_pipe[current_pipe_idx_prologue][r_a_tile_base]);
    *smem_ptr_h2_A = make_half2(__float2half(weight_val_0), __float2half(weight_val_1));
}

// Load Bsub_pipe[0] for k_tile_iter = 0

```

```

{
    int h2_idx_in_k_dim_B = threadIdx.x % NUM_H2_ELEMENTS_IN
    int shmem_k_start_for_h2_B = h2_idx_in_k_dim_B * VECTOR_

    int k_global_B_0 = k_tile_start_prologue + shmem_k_start
    int k_global_B_1 = k_tile_start_prologue + shmem_k_start

    int kw_eff_reg_B_0 = 0, kh_eff_reg_B_0 = 0, ic_eff_reg_B_0 = 0;
    bool is_valid_k_B_0 = (k_global_B_0 < K_gemm);
    if (is_valid_k_B_0) {
        kw_eff_reg_B_0 = k_global_B_0 % K_w;
        int temp_div_kw_B_0 = k_global_B_0 / K_w;
        kh_eff_reg_B_0 = temp_div_kw_B_0 % K_h;
        ic_eff_reg_B_0 = temp_div_kw_B_0 / K_h;
    }

    int kw_eff_reg_B_1 = 0, kh_eff_reg_B_1 = 0, ic_eff_reg_B_1 = 0;
    bool is_valid_k_B_1 = (k_global_B_1 < K_gemm);
    if (is_valid_k_B_1) {
        kw_eff_reg_B_1 = k_global_B_1 % K_w;
        int temp_div_kw_B_1 = k_global_B_1 / K_w;
        kh_eff_reg_B_1 = temp_div_kw_B_1 % K_h;
        ic_eff_reg_B_1 = temp_div_kw_B_1 / K_h;
    }

    int n_row_group_id_B = threadIdx.x / NUM_H2_ELEMENTS_IN
    for (int r_b_tile_base = n_row_group_id_B; r_b_tile_base < n_row_group_id_B + n_row_group_size; r_b_tile_base++) {
        float input_val_0 = 0.0f;
        if (n_params_sh[r_b_tile_base].isValidPixel && is_valid_k_B_0) {
            const NDecomposed& current_n_params = n_params_sh[r_b_tile_base];
            int h_in_eff_0 = current_n_params.h_in_base + kh_eff_reg_B_0 * K_h;
            int w_in_eff_0 = current_n_params.w_in_base + kw_eff_reg_B_0 * K_w;
            if (h_in_eff_0 >= 0 && h_in_eff_0 < H_in && w_in_eff_0 >= 0 && w_in_eff_0 < W_in) {
                input_val_0 = input_ptr[current_n_params.n_in_base + h_in_eff_0 * H_in + w_in_eff_0];
            }
        }
    }
}

```

```

    }
    float input_val_1 = 0.0f;
    if (n_params_sh[r_b_tile_base].isValidPixel && is_
        const NDecomposed& current_n_params = n_params_
        int h_in_eff_1 = current_n_params.h_in_base + k
        int w_in_eff_1 = current_n_params.w_in_base + k
        if (h_in_eff_1 >= 0 && h_in_eff_1 < H_in && w
            input_val_1 = input_ptr[current_n_params.n
                ic_eff_reg_B_1 * H
                h_in_eff_1 * W_in
                w_in_eff_1];
        }
    }
    half2* smem_ptr_h2_B = reinterpret_cast<half2*>(
        &Bsub_pipe[current_pipe_idx_prologue][r_b_tile_
    );
    *smem_ptr_h2_B = make_half2(__float2half(input_val_1
}
}
}
}

```

```

// Loop over the K_gemm dimension in tiles of WMMA_K
for (int k_tile_iter = 0; k_tile_iter < num_k_tiles; ++k_tile_it
    __syncthreads(); // Sync point for pipelining

    int compute_pipe_idx = k_tile_iter % 2;
    int load_pipe_idx = (k_tile_iter + 1) % 2;

    // --- Load Stage for next k-tile (k_tile_iter + 1) into lo
    int k_tile_start_for_load = (k_tile_iter + 1) * WMMA_K;
    if (k_tile_start_for_load < K_gemm) {
        // Load Asub_pipe[load_pipe_idx]
        {
            int h2_idx_in_k_dim_A = threadIdx.x % NUM_H2_ELEMEN
            int shmem_k_start_for_h2_A = h2_idx_in_k_dim_A * VE

            int k_global_A_0 = k_tile_start_for_load + shmem_k_

```



```

int k_global_A_1 = k_tile_start_for_load + shmem_k_

int kw_eff_reg_A_0 = 0, kh_eff_reg_A_0 = 0, ic_eff_
bool is_valid_k_A_0 = (k_global_A_0 < K_gemm);
if (is_valid_k_A_0) {
    kw_eff_reg_A_0 = k_global_A_0 % K_w;
    int temp_div_kw_A_0 = k_global_A_0 / K_w;
    kh_eff_reg_A_0 = temp_div_kw_A_0 % K_h;
    ic_eff_reg_A_0 = temp_div_kw_A_0 / K_h;
}

int kw_eff_reg_A_1 = 0, kh_eff_reg_A_1 = 0, ic_eff_
bool is_valid_k_A_1 = (k_global_A_1 < K_gemm);
if (is_valid_k_A_1) {
    kw_eff_reg_A_1 = k_global_A_1 % K_w;
    int temp_div_kw_A_1 = k_global_A_1 / K_w;
    kh_eff_reg_A_1 = temp_div_kw_A_1 % K_h;
    ic_eff_reg_A_1 = temp_div_kw_A_1 / K_h;
}

int m_row_group_id_A = threadIdx.x / NUM_H2_ELEMENT
for (int r_a_tile_base = m_row_group_id_A; r_a_tile
    int oc_idx = block_row_gemm_start + r_a_tile_base;
    float weight_val_0 = 0.0f;
    if (oc_idx < C_out && is_valid_k_A_0) {
        weight_val_0 = weight_ptr[oc_idx * C_in *
                                ic_eff_reg_A_0
                                kh_eff_reg_A_0
                                kw_eff_reg_A_0
        }
        float weight_val_1 = 0.0f;
        if (oc_idx < C_out && is_valid_k_A_1) {
            weight_val_1 = weight_ptr[oc_idx * C_in *
                                    ic_eff_reg_A_1
                                    kh_eff_reg_A_1
                                    kw_eff_reg_A_1
            }
        half2* smem_ptr_h2_A = reinterpret_cast<half2*>

```

```

        &Asub_pipe[load_pipe_idx][r_a_tile_base][sh
    );
    *smem_ptr_h2_A = make_half2(__float2half(weight
}
}

// Load Bsub_pipe[load_pipe_idx]
{
    int h2_idx_in_k_dim_B = threadIdx.x % NUM_H2_ELEMENT
    int shmem_k_start_for_h2_B = h2_idx_in_k_dim_B * VE

    int k_global_B_0 = k_tile_start_for_load + shmem_k_
    int k_global_B_1 = k_tile_start_for_load + shmem_k_

    int kw_eff_reg_B_0 = 0, kh_eff_reg_B_0 = 0, ic_eff_
    bool is_valid_k_B_0 = (k_global_B_0 < K_gemm);
    if (is_valid_k_B_0) {
        kw_eff_reg_B_0 = k_global_B_0 % K_w;
        int temp_div_kw_B_0 = k_global_B_0 / K_w;
        kh_eff_reg_B_0 = temp_div_kw_B_0 % K_h;
        ic_eff_reg_B_0 = temp_div_kw_B_0 / K_h;
    }

    int kw_eff_reg_B_1 = 0, kh_eff_reg_B_1 = 0, ic_eff_
    bool is_valid_k_B_1 = (k_global_B_1 < K_gemm);
    if (is_valid_k_B_1) {
        kw_eff_reg_B_1 = k_global_B_1 % K_w;
        int temp_div_kw_B_1 = k_global_B_1 / K_w;
        kh_eff_reg_B_1 = temp_div_kw_B_1 % K_h;
        ic_eff_reg_B_1 = temp_div_kw_B_1 / K_h;
    }

    int n_row_group_id_B = threadIdx.x / NUM_H2_ELEMENT
    for (int r_b_tile_base = n_row_group_id_B; r_b_tile
        float input_val_0 = 0.0f;
        if (n_params_sh[r_b_tile_base].isValidPixel &&
            const NDecomposed& current_n_params = n_pa
            int h_in_eff_0 = current_n_params.h_in_base

```

```

        int w_in_eff_0 = current_n_params.w_in_base;
        if (h_in_eff_0 >= 0 && h_in_eff_0 < H_in &
            input_val_0 = input_ptr[current_n_params
                ic_eff_reg_B_0
                h_in_eff_0 * V
                w_in_eff_0];

    }

}

float input_val_1 = 0.0f;
if (n_params_sh[r_b_tile_base].isValidPixel &&
    const NDecomposed& current_n_params = n_pa
    int h_in_eff_1 = current_n_params.h_in_base;
    int w_in_eff_1 = current_n_params.w_in_base;
    if (h_in_eff_1 >= 0 && h_in_eff_1 < H_in &
        input_val_1 = input_ptr[current_n_params
            ic_eff_reg_B_1
            h_in_eff_1 * V
            w_in_eff_1];

    }

}

half2* smem_ptr_h2_B = reinterpret_cast<half2*>
    &Bsub_pipe[load_pipe_idx][r_b_tile_base][sh
);
*smem_ptr_h2_B = make_half2(__float2half(input_val_1));
}

}

}

// --- Compute Stage for current k-tile (k_tile_iter) using
int a_row_start_in_tile = warp_id * WMMA_M;

wmma::fragment<wmma::matrix_a, WMMA_M, WMMA_N, WMMA_K, half,
wmma::load_matrix_sync(a_frag, &Asub_pipe[compute_pipe_idx][a

wmma::fragment<wmma::matrix_b, WMMA_M, WMMA_N, WMMA_K, half,

if (BLOCK_N_TILES_WMMA > 0) {
    int b_col_start_in_tile_current = 0 * WMMA_N;

```

```

        wmma::load_matrix_sync(b_frag_inner_pipe[0], &Bsub_pipe[0],
    }

    int current_inner_pipe_idx = 0;

    #pragma unroll
    for (int n_tile = 0; n_tile < BLOCK_N_TILES_WMMA; ++n_tile)
        int next_inner_pipe_idx = 1 - current_inner_pipe_idx;

        if (n_tile < BLOCK_N_TILES_WMMA - 1) {
            int b_col_start_in_tile_next = (n_tile + 1) * WMMA_N;
            wmma::load_matrix_sync(b_frag_inner_pipe[next_inner_pipe_idx],
        }

        wmma::mma_sync(acc_frag[n_tile], a_frag, b_frag_inner_pipe[next_inner_pipe_idx],

        current_inner_pipe_idx = next_inner_pipe_idx;
    }
}

__syncthreads();

// Store results from accumulator fragments to global memory
#pragma unroll
for (int n_tile = 0; n_tile < BLOCK_N_TILES_WMMA; ++n_tile) {
    wmma::store_matrix_sync(&C_shmem_output_buffers[warp_id][0][0], acc_frag[n_tile],

    for (int elem_idx_in_frag = lane_id; elem_idx_in_frag < WMMA_N; elem_idx_in_frag++) {
        int r_frag = elem_idx_in_frag / WMMA_N;
        int c_frag = elem_idx_in_frag % WMMA_N;

        int oc_idx = block_row_gemm_start + (warp_id * WMMA_M) + r_frag * WMMA_N + c_frag;

        int offset_in_block_N_processing = (n_tile * WMMA_N) + oc_idx;

        if (oc_idx < C_out && offset_in_block_N_processing < TILE_SIZE * TILE_SIZE) {
            n_params_sh[offset_in_block_N_processing].isValidPixel = true;
            const NDdecomposed& current_n_params = n_params_sh[offset_in_block_N_processing];
            int ow_eff = current_n_params.ow_eff;
        }
    }
}

```

```

        int oh_eff = current_n_params.oh_eff;
        int n_batch_idx = current_n_params.n_batch_idx;

        float val = C_shmem_output_buffers[warp_id][r_frag]

        if (bias_ptr != nullptr) {
            val += bias_ptr[oc_idx];
        }

        output_ptr[n_batch_idx * C_out * H_out * W_out +
                   oc_idx * H_out * W_out +
                   oh_eff * W_out +
                   ow_eff] = val;
    }
}
}
}

```

```

torch::Tensor conv2d_implicit_gemm_cuda(
    torch::Tensor input, torch::Tensor weight, torch::Tensor bias,
    int N_batch, int C_in, int H_in, int W_in,
    int C_out, int K_h, int K_w,
    int stride_h, int stride_w, int pad_h, int pad_w,
    int H_out, int W_out) {

    TORCH_CHECK(input.device().is_cuda(), "Input must be a CUDA tensor");
    TORCH_CHECK(weight.device().is_cuda(), "Weight must be a CUDA tensor");
    TORCH_CHECK(input.dtype() == torch::kFloat32, "Input must be float32");
    TORCH_CHECK(weight.dtype() == torch::kFloat32, "Weight must be float32");
    if (bias.defined()) {
        TORCH_CHECK(bias.device().is_cuda(), "Bias must be a CUDA tensor");
        TORCH_CHECK(bias.dtype() == torch::kFloat32, "Bias must be float32");
        TORCH_CHECK(bias.dim() == 1 && bias.size(0) == C_out, "Bias must be size C_out");
    }

    TORCH_CHECK(input.dim() == 4, "Input must be 4D");
    TORCH_CHECK(weight.dim() == 4, "Weight must be 4D");

```

```

TORCH_CHECK(input.size(0) == N_batch, "Input N_batch mismatch");
TORCH_CHECK(input.size(1) == C_in, "Input C_in mismatch");
TORCH_CHECK(input.size(2) == H_in, "Input H_in mismatch");
TORCH_CHECK(input.size(3) == W_in, "Input W_in mismatch");
TORCH_CHECK(weight.size(0) == C_out, "Weight C_out mismatch");
TORCH_CHECK(weight.size(1) == C_in, "Weight C_in mismatch");
TORCH_CHECK(weight.size(2) == K_h, "Weight K_h mismatch");
TORCH_CHECK(weight.size(3) == K_w, "Weight K_w mismatch");

auto output = torch::zeros({N_batch, C_out, H_out, W_out}, input

const int M_gemm = C_out;
const int N_gemm = N_batch * H_out * W_out;
const int K_gemm = C_in * K_h * K_w;

if (M_gemm == 0 || N_gemm == 0) {
    return output;
}
if (K_gemm == 0) {
    if (bias.defined()) {
        output = output + bias.reshape({1, C_out, 1, 1});
    }
    return output;
}

dim3 block_dim(THREADS_PER_BLOCK);
dim3 grid_dim(
    (N_gemm + TILE_N_PER_BLOCK - 1) / TILE_N_PER_BLOCK,
    (M_gemm + TILE_M_PER_BLOCK - 1) / TILE_M_PER_BLOCK
);

const float* bias_ptr_data = bias.defined() ? bias.data_ptr<float>() : nullptr;

cudaStream_t stream = at::cuda::getCurrentCUDAStream();
conv2d_implicit_gemm_wmma_kernel<<grid_dim, block_dim, 0, stream,
    input.data_ptr<float>(),
    weight.data_ptr<float>(),
    bias_ptr_data,

```

```

        output.data_ptr<float>(),
        N_batch, C_in, H_in, W_in,
        C_out, K_h, K_w,
        stride_h, stride_w, pad_h, pad_w,
        H_out, W_out,
        M_gemm, N_gemm, K_gemm
    );

    AT_CUDA_CHECK(cudaGetLastError());

    return output;
}
"""

conv2d_implicit_gemm_cuda_declaration = r"""
torch::Tensor conv2d_implicit_gemm_cuda(
    torch::Tensor input, torch::Tensor weight, torch::Tensor bias,
    int N_batch, int C_in, int H_in, int W_in,
    int C_out, int K_h, int K_w,
    int stride_h, int stride_w, int pad_h, int pad_w,
    int H_out, int W_out);
"""

# JIT compile the CUDA kernel
custom_conv2d_wmma_ops = load_inline(
    name="custom_conv2d_wmma_ops_optimized_k_pipe_vec_smem", # Change
    cpp_sources=conv2d_implicit_gemm_cuda_declaration,
    cuda_sources=conv2d_implicit_gemm_cuda_source,
    functions=["conv2d_implicit_gemm_cuda"],
    verbose=True,
    extra_cuda_cflags=["-arch=sm_70", "--use_fast_math", "-std=c++17"]
)

class ModelNew(nn.Module):
    def __init__(self, num_classes=1000): # num_classes is part of o
        super(ModelNew, self).__init__()

```

```

# Define Conv1 parameters (matching the original model)
self.in_channels = 3
self.out_channels = 96
self.kernel_size_val = 11 # Assuming square kernel
self.stride_val = 4        # Assuming square stride
self.padding_val = 2       # Assuming square padding

# Create a temporary Conv2d layer to initialize weights and
temp_conv = nn.Conv2d(
    in_channels=self.in_channels,
    out_channels=self.out_channels,
    kernel_size=self.kernel_size_val,
    stride=self.stride_val,
    padding=self.padding_val,
    bias=True # nn.Conv2d has bias=True by default
)
self.conv1_weight = nn.Parameter(temp_conv.weight.detach().clone())
if temp_conv.bias is not None:
    self.conv1_bias = nn.Parameter(temp_conv.bias.detach().clone())
else:
    # Correctly register 'conv1_bias' as None if not present
    self.register_parameter('conv1_bias', None)

self.custom_conv_op = custom_conv2d_wmma_ops.conv2d_implicit

def forward(self, x):
    N_batch = x.size(0)
    # C_in_runtime = x.size(1) # Should match self.in_channels
    H_in = x.size(2)
    W_in = x.size(3)

    # Calculate output dimensions
    H_out = (H_in + 2 * self.padding_val - self.kernel_size_val)
    W_out = (W_in + 2 * self.padding_val - self.kernel_size_val)

    # Bias tensor handling: pass an undefined tensor if bias is
    # The C++ TORCH_CHECK(bias.defined()) handles this by provid

```



```
        bias_tensor = self.conv1_bias if self.conv1_bias is not None

    x = self.custom_conv_op(
        x, self.conv1_weight, bias_tensor,
        N_batch, self.in_channels, H_in, W_in,
        self.out_channels, self.kernel_size_val, self.kernel_size_val,
        self.stride_val, self.stride_val, # stride_h, stride_w
        self.padding_val, self.padding_val, # pad_h, pad_w
        H_out, W_out
    )
    return x
```

CRFM is grateful to our [supporters](#).

© 2024. Stanford Center for Research on Foundation Models.

Designed by Joon Sung Park.